

Numerical Methods for Ordinary Differential Equations

Final Project — Part I

Author: Lucas Pieper-White

Course: MATH 104C - Spring 2026 **Date:** June 5th, 2026

Code: Github Link: [PiepBuilds/Math-104C-Final-Project](https://github.com/PiepBuilds/Math-104C-Final-Project)

Introduction

This project in numerical analysis aims to implement and evaluate nine numerical methods for solving the following initial value problem.

$$y' = f(t, y) \quad \text{on } [a, b] \quad \text{with } y(a) = \alpha$$

We solve the above IVP with the following methods: Euler's, Taylor's Order 2, Midpoint, Modified Euler, Heun, Runge–Kutta Order 4, Adams–Bashforth, Adams–Moulton, and the Adams Predictor–Corrector method. The primary goal is to compare accuracy, error propagation, and computational efficiency, and to connect numerical observations with theoretical error estimates.

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import approxmethods as M # the nine solvers

# --- the three initial-value problems (defined inline so the report is self-contained) ---
PROBLEM_A = { # smooth test problem
    'name': "A: y' = y - t^2 + 1",
    'f': lambda t, y: y - t**2 + 1.0,
    'dfdt': lambda t, y: y - t**2 - 2.0*t + 1.0, # total derivative f_t + f_y f
    'exact': lambda t: (t + 1.0)**2 - 0.5*np.exp(t),
    'a': 0.0, 'b': 2.0, 'alpha': 0.5,
}
PROBLEM_B = { # exponential error amplification
    'name': "B: y' = 2y",
    'f': lambda t, y: 2.0*y,
    'dfdt': lambda t, y: 4.0*y,
    'exact': lambda t: np.exp(2.0*t),
    'a': 0.0, 'b': 2.0, 'alpha': 1.0,
}
PROBLEM_C = { # rapid decay with forcing (instructor's suggested example)
    'name': "C: y' = -10y + sin t",
    'f': lambda t, y: -10.0*y + np.sin(t),
    'dfdt': lambda t, y: np.cos(t) + 100.0*y - 10.0*np.sin(t), # f_t + f_y f
    'exact': lambda t: (10.0*np.sin(t) - np.cos(t))/101.0 + (102.0/101.0)*np.exp(-10.0*t),
    'a': 0.0, 'b': 2.0, 'alpha': 1.0,
```

}

```
print('exact checks:  A:y(2)=%.6f   B:y(2)=%.6f   C:y(2)=%.6f'
      % (PROBLEM_A['exact'](2), PROBLEM_B['exact'](2), PROBLEM_C['exact'](2)))
```

exact checks: A:y(2)=5.305472 B:y(2)=54.598150 C:y(2)=0.094150

Numerical Methods

All of the methods used in this report approximate the solution on a uniform mesh defined by $t_i = a + ih$, producing approximations w_i for $y(t_i)$. We separate the methods into single-step methods and multi-step methods.

Single-Step Methods

The single-step methods (Euler, Taylor, Midpoint, Modified Euler, Heun, RK4) compute w_{i+1} using only the most recent point w_i .

Euler's Method: Euler's method takes the slope at the left endpoint and follows that tangent line to the next mesh point, repeating the tangent-line extension from each new left point. Because the solution curves away from the tangent line within each step, Euler's method has Local Truncation Error (LTE) $O(h^2)$ and Global Error $O(h)$.

$$w_{i+1} = w_i + h f(t_i, w_i)$$

Taylor's Method (Order 2): Taylor's method includes y'' through the total derivative $f' = \frac{df}{dt} = f_t + f_y f$, giving more accuracy than Euler's method, which uses only f . The cost is that the derivative f' must be computed by hand, making the method tedious and expensive compared to the others. Taylor's method of order 2 has LTE $O(h^3)$ and Global Error $O(h^2)$.

$$w_{i+1} = w_i + h \left(f(t_i, w_i) + \frac{h}{2} f'(t_i, w_i) \right)$$

Midpoint Method (RK2, Order 2): The midpoint method uses the left-endpoint slope to estimate the midpoint, then samples the slope again there to take the full step, capturing more curvature than Euler's left-endpoint slope alone. It has LTE $O(h^3)$ and Global Error $O(h^2)$, and is light on computation — only function evaluations, no derivatives.

$$w_{i+1} = w_i + h f\left(t_i + \frac{h}{2}, w_i + \frac{h}{2} f(t_i, w_i)\right)$$

Modified Euler's Method (RK2, Order 2): Modified Euler samples the slope at the left endpoint and at the predicted full-step right endpoint, then advances using the average of those two slopes. It has LTE $O(h^3)$ and Global Error $O(h^2)$.

$$w_{i+1} = w_i + \frac{h}{2} (f(t_i, w_i) + f(t_i + h, w_i + h f(t_i, w_i)))$$

Heun's Method (RK3, Order 3): Heun's method is a weighted combination of three slope evaluations — the left endpoint, an intermediate point, and a predicted point — combined with weights $\left(\frac{1}{4}, 0, \frac{3}{4}\right)$, which

cancels the error terms up to higher order. It has LTE $O(h^4)$ and Global Error $O(h^3)$.

$$w_{i+1} = w_i + \frac{h}{4}(k_1 + 3k_3)$$

$$k_1 = f(t_i, w_i), \quad k_2 = f\left(t_i + \frac{h}{3}, w_i + \frac{h}{3}k_1\right), \quad k_3 = f\left(t_i + \frac{2h}{3}, w_i + \frac{2h}{3}k_2\right)$$

Classical Runge–Kutta (RK4, Order 4): RK4 samples the slope at four points within each step — the left endpoint, two midpoint estimates, and the right endpoint — and combines them with weights $\left(\frac{1}{6}, \frac{2}{6}, \frac{2}{6}, \frac{1}{6}\right)$. This reproduces the accuracy of an order-4 Taylor method using only function evaluations, with no hand-computed derivatives. RK4 has LTE $O(h^5)$ and Global Error $O(h^4)$, making it the most accurate single-step method here, at the cost of four function evaluations per step.

$$w_{i+1} = w_i + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

$$k_1 = f(t_i, w_i), \quad k_2 = f\left(t_i + \frac{h}{2}, w_i + \frac{h}{2}k_1\right), \quad k_3 = f\left(t_i + \frac{h}{2}, w_i + \frac{h}{2}k_2\right), \quad k_4 = f(t_i + h, w_i)$$

Multi-Step Methods

The multi-step methods are started with RK4 and then reuse previously computed slopes rather than recomputing everything each step. Throughout, $f_i \equiv f(t_i, w_i)$ denotes the slope already stored from earlier steps.

Adams–Bashforth 4-Step (Explicit, Order 4): Adams–Bashforth fits an interpolating polynomial through the four most recent slopes $f_i, f_{i-1}, f_{i-2}, f_{i-3}$ and integrates it across the next step. It needs only one new function evaluation per step, which is what makes it explicit and cheap. It has Global Error $O(h^4)$. Its weakness is that it extrapolates beyond the known data, so it is comparatively inaccurate and, as we will see, prone to instability.

$$w_{i+1} = w_i + \frac{h}{24}(55f_i - 59f_{i-1} + 37f_{i-2} - 9f_{i-3})$$

Adams–Moulton 3-Step (Implicit, Order 4): Adams–Moulton again interpolates a polynomial through recent slopes, but instead of extrapolating it includes the unknown f_{i+1} , making it implicit and more stable. The unknown w_{i+1} appears on both sides through $f_{i+1} = f(t_{i+1}, w_{i+1})$, so it must be solved iteratively, which is more costly. It has Global Error $O(h^4)$.

$$w_{i+1} = w_i + \frac{h}{24}(9f_{i+1} + 19f_i - 5f_{i-1} + f_{i-2})$$

Adams Predictor–Corrector (ABM4, Order 4): The predictor–corrector method gets most of the accuracy of the implicit method at a fixed cost. It first *predicts* $w_{i+1}^{(p)}$ with the explicit Adams–Bashforth formula, then *corrects* by evaluating the slope there once and substituting it into the Adams–Moulton formula. This avoids the iterative solve while keeping order-4 accuracy at a fixed cost of two function evaluations per step. It has Global Error $O(h^4)$.

$$w_{i+1}^{(p)} = w_i + \frac{h}{24}(55f_i - 59f_{i-1} + 37f_{i-2} - 9f_{i-3})$$

$$w_{i+1} = w_i + \frac{h}{24}\left(9f(t_{i+1}, w_{i+1}^{(p)}) + 19f_i - 5f_{i-1} + f_{i-2}\right)$$

Error Metrics and Experimental Setup

Every method is run on a uniform mesh $t_i = a + ih$ for $i = 0, 1, \dots, N$, where $N = (b - a)/h$ is the number of steps and w_i denotes the method's approximation to the exact Solution $y(t_i)$, with $w_0 = \alpha$. Each problem is run at $h = 0.2, 0.1, 0.05$.

Where an exact Solution is known, the absolute Error at each Mesh point is

$$e_i = |y(t_i) - w_i|,$$

and each problems summary table includes the maximum absolute error over the mesh,

$$E(h) = \max_{0 \leq i \leq N} |y(t_i) - w_i|.$$

For Problem B the solution grows large, so we also report the relative error $|y(t_i) - w_i|/|y(t_i)|$, which measures error against the size of the Solution rather than in absolute terms.

The observed order of convergence is recovered from these maximum Errors. Since the Global Error is expected to behave like $E(h) \approx Ch^p$ for a method of Order p , halving the Step gives $E(h)/E(h/2) \approx 2^p$, thus as seen in lecture.

$$p \approx \log_2 \left(\frac{E(h)}{E(h/2)} \right).$$

This is computed only from the Error data, with no assumed value of p , so it serves as an independent check on each Method's predicted Order. The convergence plots show $E(h)$ against h on log-log axes, where this relationship makes each Method a roughly straight line of slope p .

For the stiff Problem C, a run is flagged unstable when the numerical Solution grows far beyond the size of the true Solution, specifically when $\max_i |w_i|$ exceeds ten times $\max_t |y(t)|$, which separates Methods that stay bounded from those that diverge.

Experiment helpers

Small reusable functions: run every method at a given step size, build the exact/approx/error table, and compute observed order of convergence. These are reused unchanged for all three problems.

```
In [2]: STEPS = [0.2, 0.1, 0.05]
METHODS = ['Euler', 'Taylor 2', 'Midpoint', 'Modified Euler', 'Heun (RK3)', 'RK4',
           'Adams-Bashforth 4', 'Adams-Moulton 3', 'Predictor-Corrector']

def run_all(P, h):
    f, dfdt, a, b, al = P['f'], P['dfdt'], P['a'], P['b'], P['alpha']
    return {
        'Euler': M.euler(f, a, b, al, h),
        'Taylor 2': M.taylor2(f, dfdt, a, b, al, h),
        'Midpoint': M.midpoint(f, a, b, al, h),
        'Modified Euler': M.modified_euler(f, a, b, al, h),
```

```

        'Heun (RK3)':      M.heun(f,a,b,al,h),
        'RK4':            M.rk4(f,a,b,al,h),
        'Adams-Bashforth 4': M.adams_bashforth4(f,a,b,al,h),
        'Adams-Moulton 3':  M.adams_moulton3(f,a,b,al,h),
        'Predictor-Corrector': M.predictor_corrector(f,a,b,al,h),
    }

def max_err(P, h):
    R = run_all(P, h)
    return {m: np.max(np.abs(P['exact'](t) - w)) for m,(t,w) in R.items()}

def order_table(P):
    e = {h: max_err(P, h) for h in STEPS}
    rows = []
    for m in METHODS:
        p1 = np.log2(e[0.2][m]/e[0.1][m]) if e[0.1][m] > 0 else np.nan
        p2 = np.log2(e[0.1][m]/e[0.05][m]) if e[0.05][m] > 0 else np.nan
        rows.append([m, e[0.2][m], e[0.1][m], e[0.05][m], p1, p2])
    df = pd.DataFrame(rows, columns=['method', 'max|err| h=0.2', 'max|err| h=0.1',
                                     'max|err| h=0.05', 'order .2->.1', 'order .1->.05'])

    return df

def per_mesh_df(P, h, m):
    """Burden & Faires style per-point table for one method: t, y(t_i), w_i, |error|."""
    t, w = run_all(P, h)[m]; ye = P['exact'](t)
    return pd.DataFrame({'t': t, 'y(t_i)': ye, 'w_i': w, '|error|': np.abs(ye-w)})

from IPython.display import display, HTML

_CSS = '''<style>
.mini {display:flex; flex-wrap:wrap; gap:10px; margin-bottom:14px;}
.mini .card {font-size:10px;}
.mini .cap {font-weight:600; font-size:11px; margin-bottom:2px;}
.mini table {border-collapse:collapse;}
.mini th,.mini td {border:1px solid #bbb; padding:1px 5px; text-align:right;}
</style>'''

def show_all_tables(P, h):
    """Render all nine methods' per-mesh tables in a compact grid, summary table below."""
    cards = ''
    for m in METHODS:
        tbl = per_mesh_df(P, h, m).round(6).to_html(index=False, border=0)
        cards += f'<div class="card"><div class="cap">{m}</div>{tbl}</div>'
    display(HTML(f'_{CSS}<b>Per-mesh tables at h={h}</b> (t, exact, approximation, absolute error)<br>
        f'<div class="mini">{cards}</div>'))
    oc = order_table(P).set_index('method')
    fmt = {c: '{:.3e}' for c in oc.columns[3:]}; fmt.update({'order .2->.1': '{:.2f}', 'order .1->.05': '{:.2f}'})
    display(HTML('<b>Summary: max absolute error over mesh and observed order of convergence</b>'))
    display(oc.style.format(fmt))

```

Problem A — Smooth Test Problem

$$y' = y - t^2 + 1, \quad 0 \leq t \leq 2, \quad y(0) = 0.5, \quad \text{exact: } y(t) = (t+1)^2 - \frac{1}{2}e^t.$$

This smooth problem is used to measure accuracy and observed order of convergence.

```
In [3]: # all nine per-mesh tables (compact grid) with the summary table below
show_all_tables(PROBLEM_A, 0.2)
```

Per-mesh tables at h=0.2 (t, exact, approximation, absolute error)

Euler

t	y(t_i)	w_i	error
0.0	0.500000	0.500000	0.000000
0.2	0.829299	0.800000	0.029299
0.4	1.214088	1.152000	0.062088
0.6	1.648941	1.550400	0.098541
0.8	2.127230	1.988480	0.138750
1.0	2.640859	2.458176	0.182683
1.2	3.179942	2.949811	0.230130
1.4	3.732400	3.451773	0.280627
1.6	4.283484	3.950128	0.333356
1.8	4.815176	4.428154	0.387023
2.0	5.305472	4.865785	0.439687

Taylor 2

t	y(t_i)	w_i	error
0.0	0.500000	0.500000	0.000000
0.2	0.829299	0.830000	0.000701
0.4	1.214088	1.215800	0.001712
0.6	1.648941	1.652076	0.003135
0.8	2.127230	2.132333	0.005103
1.0	2.640859	2.648646	0.007787
1.2	3.179942	3.191348	0.011406
1.4	3.732400	3.748645	0.016245
1.6	4.283484	4.306146	0.022663
1.8	4.815176	4.846299	0.031122
2.0	5.305472	5.347684	0.042212

Midpoint

t	y(t_i)	w_i	error
0.0	0.500000	0.500000	0.000000
0.2	0.829299	0.828000	0.001299
0.4	1.214088	1.211360	0.002728
0.6	1.648941	1.644659	0.004281
0.8	2.127230	2.121284	0.005945
1.0	2.640859	2.633167	0.007692
1.2	3.179942	3.170463	0.009478
1.4	3.732400	3.721165	0.011235
1.6	4.283484	4.270622	0.012862
1.8	4.815176	4.800959	0.014218
2.0	5.305472	5.290369	0.015102

Modified Euler

t	y(t_i)	w_i	error
0.0	0.500000	0.500000	0.000000
0.2	0.829299	0.826000	0.003299
0.4	1.214088	1.206920	0.007168
0.6	1.648941	1.637242	0.011698
0.8	2.127230	2.110236	0.016994
1.0	2.640859	2.617688	0.023171
1.2	3.179942	3.149579	0.030363
1.4	3.732400	3.693686	0.038714
1.6	4.283484	4.235097	0.048387
1.8	4.815176	4.755619	0.059558
2.0	5.305472	5.233055	0.072417

Heun (RK3)

t	y(t_i)	w_i	error
0.0	0.500000	0.500000	0.000000
0.2	0.829299	0.829244	0.000054
0.4	1.214088	1.213975	0.000113
0.6	1.648941	1.648766	0.000175
0.8	2.127230	2.126991	0.000239
1.0	2.640859	2.640556	0.000304
1.2	3.179942	3.179576	0.000365
1.4	3.732400	3.731980	0.000420
1.6	4.283484	4.283023	0.000461
1.8	4.815176	4.814697	0.000480
2.0	5.305472	5.305007	0.000465

RK4

t	y(t_i)	w_i	error
0.0	0.500000	0.500000	0.000000
0.2	0.829299	0.829293	0.000005
0.4	1.214088	1.214076	0.000011
0.6	1.648941	1.648922	0.000019
0.8	2.127230	2.127203	0.000027
1.0	2.640859	2.640823	0.000036
1.2	3.179942	3.179894	0.000047
1.4	3.732400	3.732340	0.000060
1.6	4.283484	4.283409	0.000074
1.8	4.815176	4.815086	0.000091
2.0	5.305472	5.305363	0.000109

Adams-Bashforth 4

t	y(t_i)	w_i	error
0.0	0.500000	0.500000	0.000000
0.2	0.829299	0.829293	0.000005
0.4	1.214088	1.214076	0.000011
0.6	1.648941	1.648922	0.000019
0.8	2.127230	2.127289	0.000060
1.0	2.640859	2.641053	0.000194
1.2	3.179942	3.180314	0.000373
1.4	3.732400	3.733019	0.000619
1.6	4.283484	4.284442	0.000959
1.8	4.815176	4.816596	0.001419
2.0	5.305472	5.307508	0.002036

Adams-Moulton 3

t	y(t_i)	w_i	error
0.0	0.500000	0.500000	0.000000
0.2	0.829299	0.829293	0.000005
0.4	1.214088	1.214076	0.000011
0.6	1.648941	1.648920	0.000021
0.8	2.127230	2.127196	0.000033
1.0	2.640859	2.640809	0.000050
1.2	3.179942	3.179868	0.000073
1.4	3.732400	3.732296	0.000104
1.6	4.283484	4.283338	0.000145
1.8	4.815176	4.814977	0.000199
2.0	5.305472	5.305202	0.000270

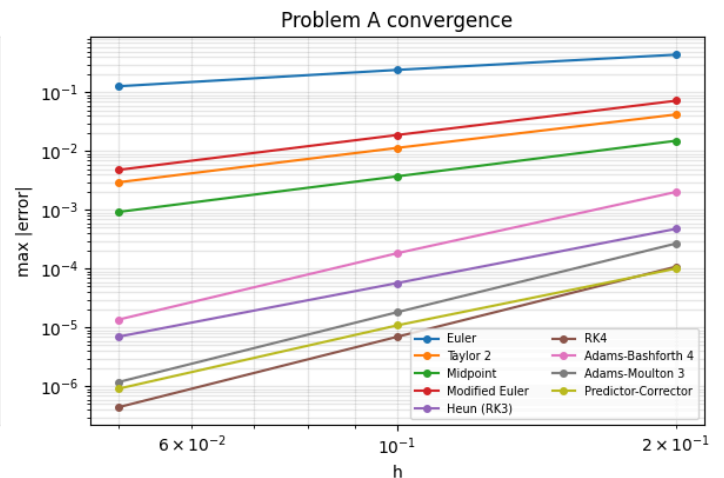
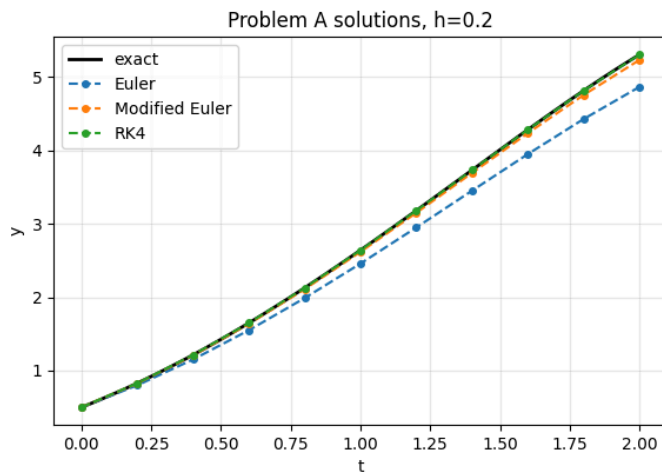
Predictor-Corrector

t	y(t_i)	w_i	error
0.0	0.500000	0.500000	0.000000
0.2	0.829299	0.829293	0.000005
0.4	1.214088	1.214076	0.000011
0.6	1.648941	1.648922	0.000019
0.8	2.127230	2.127206	0.000024
1.0	2.640859	2.640829	0.000030
1.2	3.179942	3.179903	0.000039
1.4	3.732400	3.732350	0.000050
1.6	4.283484	4.283421	0.000063
1.8	4.815176	4.815096	0.000080
2.0	5.305472	5.305371	0.000101

Summary: max absolute error over mesh and observed order of convergence

	max err h=0.2	max err h=0.1	max err h=0.05	order .2->.1	order .1->.05
method					
Euler	4.397e-01	2.420e-01	1.275e-01	0.86	0.92
Taylor 2	4.221e-02	1.141e-02	2.964e-03	1.89	1.94
Midpoint	1.510e-02	3.747e-03	9.277e-04	2.01	2.01
Modified Euler	7.242e-02	1.890e-02	4.820e-03	1.94	1.97
Heun (RK3)	4.797e-04	5.734e-05	7.015e-06	3.06	3.03
RK4	1.089e-04	6.990e-06	4.421e-07	3.96	3.98
Adams-Bashforth 4	2.036e-03	1.846e-04	1.368e-05	3.46	3.75
Adams-Moulton 3	2.703e-04	1.831e-05	1.184e-06	3.88	3.95
Predictor-Corrector	1.013e-04	1.093e-05	9.132e-07	3.21	3.58

```
In [4]: # solution overlay + convergence
fig, ax = plt.subplots(1, 2, figsize=(12,4.3))
tt = np.linspace(0,2,400); ax[0].plot(tt, PROBLEM_A['exact'](tt),'k-',lw=2,label='exact')
for m in ['Euler','Modified Euler','RK4']:
    t,w = run_all(PROBLEM_A,0.2)[m]; ax[0].plot(t,w,'o--',ms=4,label=m)
ax[0].set(title='Problem A solutions, h=0.2', xlabel='t', ylabel='y'); ax[0].legend(); ax[0].gr
e = {h: max_err(PROBLEM_A,h) for h in STEPS}
for m in METHODS: ax[1].loglog(STEPS,[e[h][m] for h in STEPS],'o-',ms=4,label=m)
ax[1].set(title='Problem A convergence', xlabel='h', ylabel='max |error|'); ax[1].legend(fontsi
plt.tight_layout(); plt.show()
```



Problem A: Smooth Test Problem

Hand Verification (Euler, $h = 0.2$). First we check code the implementation, the first Euler steps computed by hand from $w_{i+1} = w_i + hf(t_i, w_i)$ with $f = y - t^2 + 1$ give $w_1 = 0.8$, $w_2 = 1.152$, $w_3 = 1.5504$, matching our output exactly. The exact value $y(2) = (2+1)^2 - \frac{1}{2}e^2 = 5.305472$ also matches the table, confirming both the approximations and the reported errors.

Results Discussion: Across all nine Methods the measured order matches up with the predicted order almost exactly. At $h = 0.2$ the max Global Error runs from 4.4×10^{-1} (Euler) down to 1.1×10^{-4} (RK4), roughly a 10^3 spread between the worst, Eulers Method the and best, Runge-Kutta 4 Method all on the same mesh.

Following what we learned in lecture, halving h should scale the Error by about $\frac{1}{2}^p$, so the measured order is recovered as $p \approx \log_2\left(\frac{\text{Error}(h)}{\text{Error}(h/2)}\right)$. For Euler, the error falls from 4.397×10^{-1} to 2.420×10^{-1} as h goes 0.2 to 0.1, a ratio of only 1.82, giving $\log_2(1.82) = 0.86 \approx 1$: halving h barely helps, exactly what an Order-1 Method does. For RK4 under the same refining the Error drops from 1.089×10^{-4} to 6.990×10^{-6} , a ratio of $15.6 \approx 2^4$, giving $\log_2(15.6) = 3.96 \approx 4$. In the convergence plot, we can also see the Euler and RK4 methods as our best and worst methods forming upper and lower bounds for the other methods. Comparing the order calculations in the above summary table to the predicted convergence orders defined in the methods section we find that Taylor 2, Midpoint, and Modified Euler all are close to 2, Midpoint almost exact at 2.01, and Heun measures $3.06, 3.03 \approx 3$. Since each measured exponent is built only from the Error data and still settles onto the integer order predicted in the methods section, this is confirmation that the global Error behaves like $O(h^p)$ as predicted.

The three multistep Methods are not so exact, they land slightly below 4 as h goes from 0.2 to 0/1. Adams–Bashforth comes in at 3.46, and Predictor–Corrector at 3.21. Despite this, they get closer to 4 as h shrinks again 3.75 and 3.58. This is feature is somewhat expected given how we built them since each is started by RK4 for its first few Steps, so a little Error in the beginning carries on and drags the measured exponent under 4 at the coarser step sizes but coming back under refinement.

The reason RK4 dominates Eulers in Problem A is because of how they calculate slope and project forward. Euler uses only the left endpoint slope to move forward the entire step, so any deviation from that slope inside the interval is ignored and feeds straight into the Local Truncation Error, leaving Global Error at $O(h)$. RK4 samples four slopes per step; the left endpoint, two midpoints, and the right endpoint, then takes a weighted average and capturing curvature and retaining global Error $O(h^4)$. RK4 pays for this with four function evaluations per Step against Euler's one, but on a smooth problem the high order lets us hit a target accuracy in far fewer, larger steps, winning by orders of magnitude anyway.

Problem B — Error Amplification

$$y' = 2y, \quad 0 \leq t \leq 2, \quad y(0) = 1, \quad \text{exact: } y(t) = e^{2t}.$$

Because the true solution grows exponentially, we expect early errors to be amplified over time.

```
In [5]: # all nine per-mesh tables (compact grid) with the summary table below
show_all_tables(PROBLEM_B, 0.2)
```


Per-mesh tables at h=0.2 (t, exact, approximation, absolute error)

Euler

t	y(t_i)	w_i	error
0.0	1.000000	1.000000	0.000000
0.2	1.491825	1.400000	0.091825
0.4	2.225541	1.960000	0.265541
0.6	3.320117	2.744000	0.576117
0.8	4.953032	3.841600	1.111432
1.0	7.389056	5.378240	2.010816
1.2	11.023176	7.529536	3.493640
1.4	16.444647	10.541350	5.903296
1.6	24.532530	14.757891	9.774640
1.8	36.598234	20.661047	15.937188
2.0	54.598150	28.925465	25.672685

Taylor 2

t	y(t_i)	w_i	error
0.0	1.000000	1.000000	0.000000
0.2	1.491825	1.480000	0.011825
0.4	2.225541	2.190400	0.035141
0.6	3.320117	3.241792	0.078325
0.8	4.953032	4.797852	0.155180
1.0	7.389056	7.100821	0.288235
1.2	11.023176	10.509215	0.513961
1.4	16.444647	15.553639	0.891008
1.6	24.532530	23.019385	1.513145
1.8	36.598234	34.068690	2.529544
2.0	54.598150	50.421662	4.176488

Midpoint

t	y(t_i)	w_i	error
0.0	1.000000	1.000000	0.000000
0.2	1.491825	1.480000	0.011825
0.4	2.225541	2.190400	0.035141
0.6	3.320117	3.241792	0.078325
0.8	4.953032	4.797852	0.155180
1.0	7.389056	7.100821	0.288235
1.2	11.023176	10.509215	0.513961
1.4	16.444647	15.553639	0.891008
1.6	24.532530	23.019385	1.513145
1.8	36.598234	34.068690	2.529544
2.0	54.598150	50.421662	4.176488

Modified Euler

t	y(t_i)	w_i	error
0.0	1.000000	1.000000	0.000000
0.2	1.491825	1.480000	0.011825
0.4	2.225541	2.190400	0.035141
0.6	3.320117	3.241792	0.078325
0.8	4.953032	4.797852	0.155180
1.0	7.389056	7.100821	0.288235
1.2	11.023176	10.509215	0.513961
1.4	16.444647	15.553639	0.891008
1.6	24.532530	23.019385	1.513145
1.8	36.598234	34.068690	2.529544
2.0	54.598150	50.421662	4.176488

Heun (RK3)

t	y(t_i)	w_i	error
0.0	1.000000	1.000000	0.000000
0.2	1.491825	1.490667	0.001158
0.4	2.225541	2.222087	0.003454
0.6	3.320117	3.312391	0.007726
0.8	4.953032	4.937671	0.015361
1.0	7.389056	7.360422	0.028634
1.2	11.023176	10.971935	0.051241
1.4	16.444647	16.355498	0.089148
1.6	24.532530	24.380596	0.151934
1.8	36.598234	36.343342	0.254892
2.0	54.598150	54.175809	0.422341

RK4

t	y(t_i)	w_i	error
0.0	1.000000	1.000000	0.000000
0.2	1.491825	1.491733	0.000091
0.4	2.225541	2.225268	0.000273
0.6	3.320117	3.319507	0.000610
0.8	4.953032	4.951819	0.001213
1.0	7.389056	7.386794	0.002262
1.2	11.023176	11.019126	0.004050
1.4	16.444647	16.437598	0.007049
1.6	24.532530	24.520513	0.012017
1.8	36.598234	36.578067	0.020168
2.0	54.598150	54.564722	0.033429

Adams-Bashforth 4

t	y(t_i)	w_i	error
0.0	1.000000	1.000000	0.000000
0.2	1.491825	1.491733	0.000091
0.4	2.225541	2.225268	0.000273

Adams-Moulton 3

t	y(t_i)	w_i	error
0.0	1.000000	1.000000	0.000000
0.2	1.491825	1.491733	0.000091
0.4	2.225541	2.225268	0.000273

t	y(t_i)	w_i	error
0.6	3.320117	3.319507	0.000610
0.8	4.953032	4.944110	0.008922
1.0	7.389056	7.360518	0.028538
1.2	11.023176	10.959190	0.063986
1.4	16.444647	16.318214	0.126433
1.6	24.532530	24.297409	0.235122
1.8	36.598234	36.177879	0.420355
2.0	54.598150	53.867503	0.730647

t	y(t_i)	w_i	error
0.6	3.320117	3.320344	0.000227
0.8	4.953032	4.954364	0.001331
1.0	7.389056	7.392515	0.003459
1.2	11.023176	11.030534	0.007358
1.4	16.444647	16.458902	0.014255
1.6	24.532530	24.558688	0.026157
1.8	36.598234	36.644556	0.046321
2.0	54.598150	54.678144	0.079994

Predictor-Corrector

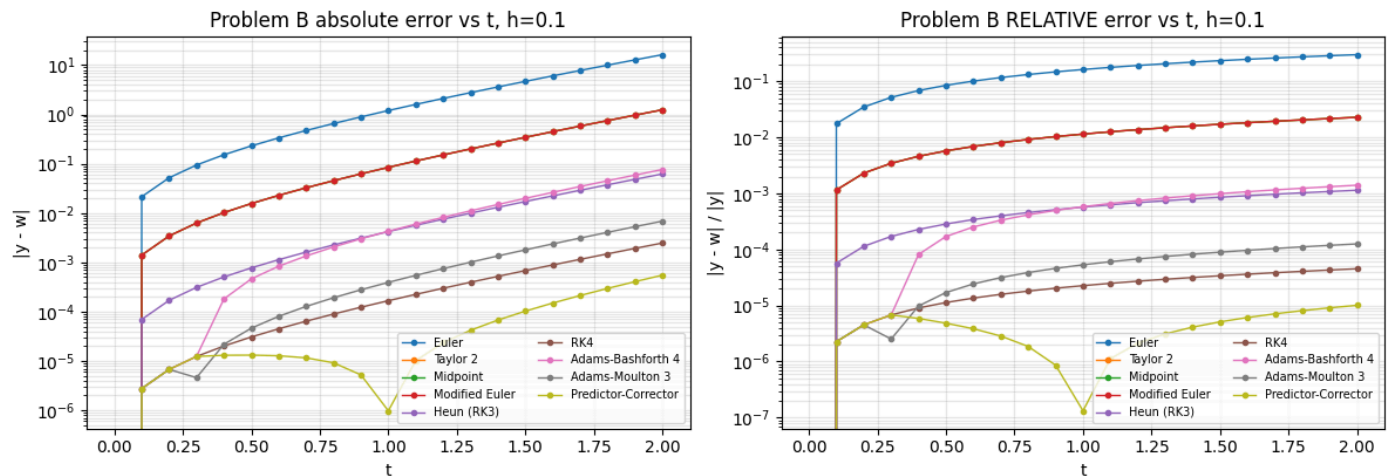
t	y(t_i)	w_i	error
0.0	1.000000	1.000000	0.000000
0.2	1.491825	1.491733	0.000091
0.4	2.225541	2.225268	0.000273
0.6	3.320117	3.319507	0.000610
0.8	4.953032	4.951724	0.001309
1.0	7.389056	7.386499	0.002557
1.2	11.023176	11.018463	0.004713
1.4	16.444647	16.436275	0.008372
1.6	24.532530	24.518042	0.014488
1.8	36.598234	36.573638	0.024597
2.0	54.598150	54.557007	0.041143

Summary: max absolute error over mesh and observed order of convergence

	max err h=0.2	max err h=0.1	max err h=0.05	order .2->.1	order .1->.05
method					
Euler	2.567e+01	1.626e+01	9.339e+00	0.66	0.80
Taylor 2	4.176e+00	1.241e+00	3.367e-01	1.75	1.88
Midpoint	4.176e+00	1.241e+00	3.367e-01	1.75	1.88
Modified Euler	4.176e+00	1.241e+00	3.367e-01	1.75	1.88
Heun (RK3)	4.223e-01	6.203e-02	8.401e-03	2.77	2.88
RK4	3.343e-02	2.466e-03	1.675e-04	3.76	3.88
Adams-Bashforth 4	7.306e-01	7.630e-02	6.045e-03	3.26	3.66
Adams-Moulton 3	7.999e-02	6.850e-03	4.977e-04	3.55	3.78
Predictor-Corrector	4.114e-02	5.501e-04	2.443e-04	6.22	1.17

```
In [6]: # absolute error grows, but relative error stays bounded
fig, ax = plt.subplots(1, 2, figsize=(12,4.3))
R = run_all(PROBLEM_B,0.1)
for m in METHODS:
    t,w = R[m]; ax[0].semilogy(t, np.abs(PROBLEM_B['exact'](t)-w), 'o-', ms=3, lw=1, label=m)
ax[0].set(title='Problem B absolute error vs t, h=0.1', xlabel='t', ylabel='|y - w|'); ax[0].le
for m in METHODS:
```

```
t,w = R[m]; ye = PROBLEM_B['exact'](t); ax[1].semilogy(t, np.abs((ye-w)/ye),'o-',ms=3,lw=1,
ax[1].set(title='Problem B RELATIVE error vs t, h=0.1', xlabel='t', ylabel='|y - w| / |y|'); ax
plt.tight_layout(); plt.show()
```



Problem B: Error Amplification

In Problem B the exact solution $y = e^{2t}$ reaches 54.60 at $t = 2$ so absolute errors are large by construction. At $h = 0.2$ Euler gives $w(2) = 28.93$ vs exact 54.60, an absolute error of 25.67, large because the solution is large. The relative error $\frac{|y-w|}{|y|}$ is more meaningful: for Euler it is (0.47, 0.30, 0.17) across $h = 0.2, 0.1, 0.05$, and for RK4 we have 6.1×10^{-4} down to 3.1×10^{-6} . Relative Error stays bounded and shrinks with h , so the Methods are not failing. As noted in the methods section, the Global Error is the accumulation of each Step's Local Truncation Error; here each step's error is carried forward and amplified by the same exponential growth as the true solution, so absolute Error grows while relative Error stays under control. This is error propagation in a growing problem. The problem is still well-posed however, the errors are amplified but stay bounded relative to the solution; small changes in the problem lead to small changes in the solution.

Taylor's order 2, Midpoint, and Modified Euler give identical errors at every step size coming in at (4.176, 1.241, 0.337). Comparing the iteration schemes outlined in the methods section, for the linear problem $y' = 2y$ all three order 2 methods reduce to the same update which is the amplification factor $(1 + 2h + 2h^2)$; their distinct slope samplings (Midpoint's half-step, Modified Euler's endpoint average) collapse to the same recurrence once $f = 2y$ is substituted. We see that they are not only the same order, they are the same method. This is the concrete example where the order exponent p is shared but the problem itself removes the leading constant.

Also notice in the plot and summary table that the measured orders trend to their predicted values but slower than in Problem A: Euler 0.66, 0.80 toward 1, the order 2 trio 1.75, 1.88 toward 2, RK4 3.76, 3.88 toward 4. Because the solution is growing, the measured orders approach their integer values more slowly than in Problem A.

Problem C — Rapid Decay with Forcing (Stability)

$$y' = -10y + \sin t, \quad 0 \leq t \leq 2, \quad y(0) = 1.$$

This is a linear first-order ODE, so it has a closed-form solution. Using the integrating factor e^{10t} ,

$$y(t) = \frac{10 \sin t - \cos t}{101} + \frac{102}{101} e^{-10t}.$$

I chose this problem to study stability rather than accuracy. The $-10y$ term makes the Solution decay rapidly from 1 before settling into a small oscillation driven by $\sin t$. For an explicit Method the key quantity is $h\lambda = -10h$, and forward Euler is stable only when $|1 + h\lambda| \leq 1$, that is $h \leq 2/10 = 0.2$. So I expected the explicit Methods to handle $h = 0.2$ poorly and to improve sharply as h shrinks.

```
In [7]: # all nine per-mesh tables (compact grid) with the summary table below
show_all_tables(PROBLEM_C, 0.2)

# stability flag: a run is UNSTABLE if it grows far beyond the true solution's size
yref = np.max(np.abs(PROBLEM_C['exact'])(np.linspace(PROBLEM_C['a'], PROBLEM_C['b'], 400))))
rows = []
for h in STEPS:
    R = run_all(PROBLEM_C, h)
    rows.append([h] + ['stable' if np.max(np.abs(w)) <= 10*yref else 'UNSTABLE'
                      for m in METHODS for (t,w) in [R[m]]])
display(HTML('<b>Stability: which methods stay bounded at each step size</b>'))
display(pd.DataFrame(rows, columns=['h']+METHODS).set_index('h').T)
print('Euler stable iff h <= 2/10 = 0.2')
```

Per-mesh tables at h=0.2 (t, exact, approximation, absolute error)

Euler

t	y(t_i)	w_i	error
0.0	1.000000	1.000000	0.000000
0.2	0.146642	-1.000000	1.146642
0.4	0.047934	1.039734	0.991800
0.6	0.050237	-0.961850	1.012087
0.8	0.064466	1.074779	1.010313
1.0	0.078010	-0.931307	1.009318
1.2	0.088700	1.099602	1.010902
1.4	0.095887	-0.913194	1.009081
1.6	0.099257	1.110284	1.011027
1.8	0.098670	-0.910369	1.009039
2.0	0.094150	1.105139	1.010989

Taylor 2

t	y(t_i)	w_i	error
0.0	1.000000	1.000000	0.000000
0.2	0.146642	1.020000	0.873358
0.4	0.047934	1.039601	0.991667
0.6	0.050237	1.058023	1.007786
0.8	0.064466	1.074529	1.010063
1.0	0.078010	1.088463	1.010453
1.2	0.088700	1.099269	1.010570
1.4	0.095887	1.106517	1.010629
1.6	0.099257	1.109916	1.010659
1.8	0.098670	1.109332	1.010662
2.0	0.094150	1.104788	1.010638

Midpoint

t	y(t_i)	w_i	error
0.0	1.000000	1.000000	0.000000
0.2	0.146642	1.019967	0.873325
0.4	0.047934	1.039337	0.991403
0.6	0.050237	1.057338	1.007101
0.8	0.064466	1.073253	1.008787
1.0	0.078010	1.086448	1.008437
1.2	0.088700	1.096395	1.007695
1.4	0.095887	1.102699	1.006811
1.6	0.099257	1.105108	1.005851
1.8	0.098670	1.103526	1.004856
2.0	0.094150	1.098016	1.003867

Modified Euler

t	y(t_i)	w_i	error
0.0	1.000000	1.000000	0.000000
0.2	0.146642	1.019867	0.873225
0.4	0.047934	1.038942	0.991008
0.6	0.050237	1.056464	1.006227
0.8	0.064466	1.071736	1.007270
1.0	0.078010	1.084147	1.006137
1.2	0.088700	1.093204	1.004504
1.4	0.095887	1.098545	1.002658
1.6	0.099257	1.099957	1.000700
1.8	0.098670	1.097385	0.998715
2.0	0.094150	1.090930	0.996780

Heun (RK3)

t	y(t_i)	w_i	error
0.0	1.000000	1.000000	0.000000
0.2	0.146642	-0.326716	0.473358
0.4	0.047934	0.141704	0.093770
0.6	0.050237	0.010437	0.039799
0.8	0.064466	0.076767	0.012301
1.0	0.078010	0.074032	0.003978
1.2	0.088700	0.090348	0.001648
1.4	0.095887	0.095727	0.000160
1.6	0.099257	0.099735	0.000478
1.8	0.098670	0.098952	0.000282
2.0	0.094150	0.094495	0.000345

RK4

t	y(t_i)	w_i	error
0.0	1.000000	1.000000	0.000000
0.2	0.146642	0.346611	0.199969
0.4	0.047934	0.141597	0.093663
0.6	0.050237	0.085001	0.034765
0.8	0.064466	0.076372	0.011906
1.0	0.078010	0.081816	0.003806
1.2	0.088700	0.089705	0.001005
1.4	0.095887	0.095919	0.000032
1.6	0.099257	0.098944	0.000313
1.8	0.098670	0.098235	0.000435
2.0	0.094150	0.093680	0.000470

Adams-Bashforth 4

t	y(t_i)	w_i	error
0.0	1.000000	1.000000	0.000000
0.2	0.146642	0.346611	0.199969
0.4	0.047934	0.141597	0.093663

Adams-Moulton 3

t	y(t_i)	w_i	error
0.0	1.000000	1.000000	0.000000
0.2	0.146642	0.346611	0.199969
0.4	0.047934	0.141597	0.093663

t	y(t_i)	w_i	error
0.6	0.050237	0.085001	0.034765
0.8	0.064466	0.201464	0.136998
1.0	0.078010	-0.324279	0.402289
1.2	0.088700	2.174509	2.085810
1.4	0.095887	-9.751498	9.847385
1.6	0.099257	46.984227	46.884970
1.8	0.098670	-223.055064	223.153734
2.0	0.094150	1062.173265	1062.079115

t	y(t_i)	w_i	error
0.6	0.050237	0.042410	0.007827
0.8	0.064466	0.076576	0.012110
1.0	0.078010	0.067207	0.010804
1.2	0.088700	0.095497	0.006798
1.4	0.095887	0.090465	0.005423
1.6	0.099257	0.103196	0.003940
1.8	0.098670	0.095742	0.002928
2.0	0.094150	0.096322	0.002172

Predictor-Corrector

t	y(t_i)	w_i	error
0.0	1.000000	1.000000	0.000000
0.2	0.146642	0.346611	0.199969
0.4	0.047934	0.141597	0.093663
0.6	0.050237	0.085001	0.034765
0.8	0.064466	-0.041935	0.106401
1.0	0.078010	-0.206436	0.284447
1.2	0.088700	-0.142829	0.231528
1.4	0.095887	0.281566	0.185678
1.6	0.099257	0.672914	0.573657
1.8	0.098670	0.342291	0.243621
2.0	0.094150	-0.725426	0.819576

Summary: max absolute error over mesh and observed order of convergence

max|err| h=0.2 max|err| h=0.1 max|err| h=0.05 order .2->.1 order .1->.05

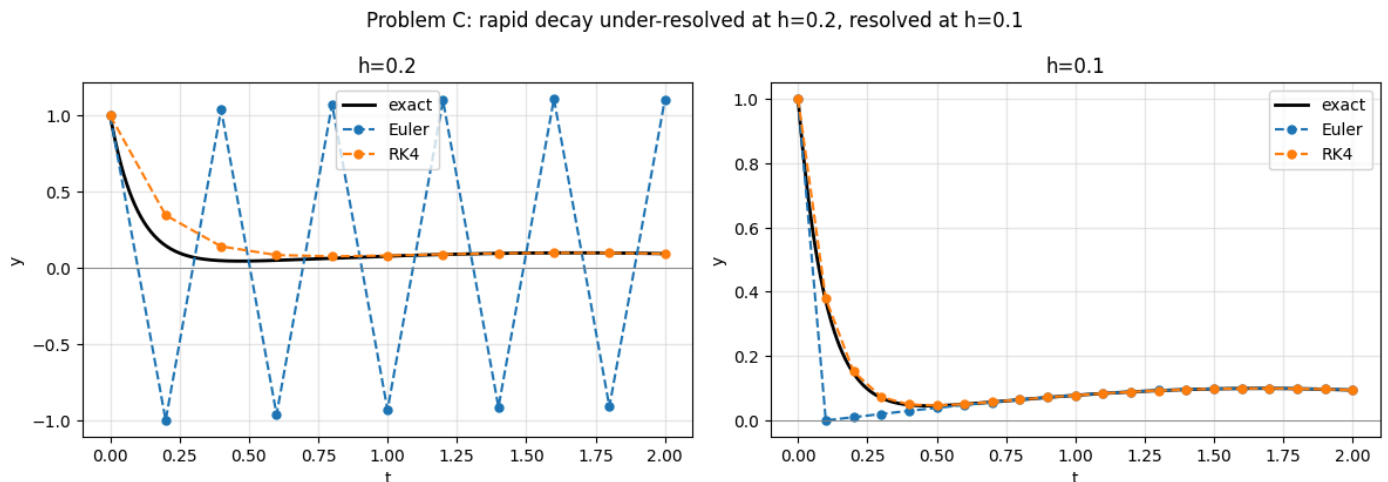
method					
Euler	1.147e+00	3.716e-01	1.191e-01	1.63	1.64
Taylor 2	1.011e+00	1.334e-01	2.297e-02	2.92	2.54
Midpoint	1.009e+00	1.334e-01	2.297e-02	2.92	2.54
Modified Euler	1.007e+00	1.334e-01	2.297e-02	2.92	2.54
Heun (RK3)	4.734e-01	3.489e-02	2.891e-03	3.76	3.59
RK4	2.000e-01	7.192e-03	2.943e-04	4.80	4.61
Adams-Bashforth 4	1.062e+03	8.368e+04	6.230e+02	-6.30	7.07
Adams-Moulton 3	2.000e-01	7.192e-03	2.943e-04	4.80	4.61
Predictor-Corrector	8.196e-01	2.416e-02	1.179e-03	5.08	4.36

Stability: which methods stay bounded at each step size

	h	0.20	0.10	0.05
Euler		stable	stable	stable
Taylor 2		stable	stable	stable
Midpoint		stable	stable	stable
Modified Euler		stable	stable	stable
Heun (RK3)		stable	stable	stable
RK4		stable	stable	stable
Adams-Bashforth 4		UNSTABLE	UNSTABLE	UNSTABLE
Adams-Moulton 3		stable	stable	stable
Predictor-Corrector		stable	stable	stable

Euler stable iff $h \leq 2/10 = 0.2$

```
In [8]: # the fast decay is under-resolved at h=0.2 but well-resolved at h=0.1
fig, ax = plt.subplots(1, 2, figsize=(12,4.3)); tt = np.linspace(0,2,400)
for j,h in enumerate([0.2,0.1]):
    R = run_all(PROBLEM_C,h); ax[j].plot(tt, PROBLEM_C['exact'](tt), 'k-', lw=2, label='exact')
    for m in ['Euler', 'RK4']:
        t,w = R[m]; ax[j].plot(t,w, 'o--', ms=5, label=m)
    ax[j].axhline(0, color='gray', lw=.6); ax[j].set(title=f'h={h}', xlabel='t', ylabel='y'); ax[j]
fig.suptitle('Problem C: rapid decay under-resolved at h=0.2, resolved at h=0.1'); plt.tight_la
```



Problem C: Rapid Decay with Forcing

In Problem C, we are testing stability. The $-10y$ term gives a decay timescale of $1/10$, so at $h = 0.2$ even RK4 has an Error of 0.200, versus 1.1×10^{-4} on Problem A at the same h . A step that large cannot capture the fast decay, so the same method that was accurate on Problem A is not here.

Euler shows the stability bound well. Taking the Euler scheme from the methods section, $w_{i+1} = w_i + hf_i$, and substituting $f = \lambda y$ gives $w_{i+1} = (1 + h\lambda)w_i$, so the step is stable only when $|1 + h\lambda| \leq 1$, i.e. $h \leq 0.2$. At $h = 0.2$ the factor is $1 + (0.2)(-10) = -1$, so the solution oscillates instead of decaying as

seen in the plotted blue line. We see that as the first steps are $(1, -1, 1.04, -0.962, 1.075)$, giving max Error 1.147. At $h = 0.1$ the factor is 0, inside the stable region, and the Error drops to 0.372.

Adams–Bashforth is never stable, blowing up at every step size with max Errors $1.06 \times 10^3, 8.37 \times 10^4, 623$ at $h = 0.2, 0.1, 0.05$. As an explicit extrapolating method, its stability region is not big enough to contain $h\lambda = -10h$ even at the smallest step h . The Predictor–Corrector uses that same AB4 formula as its predictor and the implicit Adams–Moulton corrects, brining each step back into to the stable region after the explicit predictor oversteps, overall staying stable (0.820, 0.0242, 0.00118) in problem C.

RK4 and Adams–Moulton report identical errors (0.200, 0.00719, 0.000294) as AM3 is first kicked off by RK4 and the worst error sits in that shared early transient. The measured Orders are erratic and inflated, with RK4 at 4.80 and the Order-2 trio at 2.92, because the h values are still too large for the clean $E \approx Ch^p$ behavior to appear; the error formula only holds once h is small enough, and the fast decay here means $h = 0.2$ is nowhere near that point. Thus for a stiff problem, we choose to analyze stability over order.

Error and Stability Analysis

The basis across all three problems is the error chain created when each step incurs Local Truncation Error which accumulate over N steps into the Global Error that is one degree lower. Problem A confirmed this directly as every measured order landing on the theoretical integer order for its method, validating both the our approximation scheme code and the $O(h^p)$ theory. In Problems B and C, we see that order describes method accuracy under only two conditions: that h is small enough, and that the method is stable.

In problem B we tested the first; whether or not h is small enough. Masured orders approached their integers rather slowly because the growing solution inflates the errors and kept $h = 0.2$ from being small enough for clean $E \approx Ch^p$ behavior. It also showed that the order is just the exponent p in $E \approx Ch^p$, not the full size of the Error. Taylor 2, Midpoint, and Modified Euler are all Order 2, but on $y' = 2y$ they reduced to the same formula giving identical errors. With that said, we have shown that order tells you how fast the error shrinks as h shrinks, but not whether two methods of the same order will actually behave the same is dependent on the problem type.

Problem C tested the second, stability, which is a separate axis from order. Substituting $f = \lambda y$ turns each method into multiplication by an amplification factor, and the Method stays bounded only when that factor is at most one in magnitude which is the set of $h\lambda$ where this holds is the stability region. As shown in problem C, high order doesn't gauarentee accuracy alone, evident by the unstable Adams–Bashforth Order 4 where the region is not big enough to contain $h\lambda = -10h$ thus diverging at every Step size. In contrast the Predictor–Corrector Method built on that same AB4 predictor stayed stable because of its implicit corrector AM3.

On smooth Problem A the ranking is purely by Order; on stiff Problem C that ranking is replaced b stability as the deciding factor. Thus a method's behavior is not always fixed and depends on the interaction between method and problem. Order governs accuracy in the stable, small h regime, but stability governs whether a usable answer exists at all, and which dominates depends on the differential equation at hand.

Conclusions

In summary, the nine Methods trade off along three axes of comparison: accuracy, stability, and computational cost. Cost is measured in function evaluations per Step: Euler uses 1, the RK2 Methods including Midpoint and Modified Euler use 2, Heun uses 3, and RK4 with 4. The multi-step reuse stored slopes from RK4 and need only about 1 evaluation per step for Adams-bashforth and Adams Moulton, and 2 for Predictor-Corrector.

On the smooth problem these tradeoffs favor high order as the deciding factor. With that in mind, notice that RK4 costs four times Euler's per-step, but its $O(h^4)$ accuracy lets it reach a target Error in far fewer, larger Steps, and thus winning overall. The multistep Methods are the efficiency case, matching RK4's Order 4 at roughly one evaluation per step by recycling past Slopes, at the price of needing RK4 to start all of them and being more sensitive to stability.

We see that sensitivity well in the stiff Problem C where accuracy and cost don't matter until stability is satisfied. The cheap, high order Adams-Bashforth method was the worst performer because $h\lambda$ left its stability region at all step sizes, and only the methods that stayed bounded gave usable answers. In conclusion, in problems A, B and C, we have shown that not a single method is best, rather the method selection depends on whether the problem is smooth or stiff, and on which of accuracy, stability, and cost matters most for that problem.